

Justificación del css

1. Reset básico

```
* {  
    margin: 0;  
    padding: 0;  
    box-sizing: border-box;  
}
```

Este bloque se usa para eliminar los márgenes y rellenos automáticos que traen los navegadores por defecto. Así se empieza desde una base más limpia y controlada.

La propiedad `box-sizing: border-box` facilita trabajar con el modelo de caja, ya que hace que el ancho y alto del elemento incluyan el borde y el relleno. Esto se relaciona directamente con el box model visto en clase.

2. Estilos generales de página

```
html, body {  
    width: 100%;  
    height: 100%;  
    font-family: Arial, Helvetica, sans-serif;  
    overflow-x: hidden;  
}  
  
body {  
    background-color: #000000;  
    color: #ffffff;  
}
```

Aquí se definen estilos generales para toda la página.

- `width: 100%` y `height: 100%` ayudan a que la página ocupe todo el espacio disponible.
 - `font-family` establece la tipografía general del documento, algo que también aparece en los ejercicios de clase como una propiedad básica de texto.
 - `overflow-x: hidden` evita que aparezca una barra horizontal si algún elemento sobresale un poco.
 - `background-color` y `color` definen el color de fondo general y el color del texto.
-

3. Regla general para imágenes y vídeos

```
img, video {  
  max-width: 100%;  
  display: block;  
}
```

Este bloque hace que las imágenes y vídeos no sobresalgan del contenedor y se adapten mejor al ancho disponible.

Además, `display: block` evita pequeños espacios que a veces aparecen cuando se tratan como elementos en línea. Esto se puede justificar con lo trabajado sobre elementos de bloque y en línea.

4. Cabecera principal

```
.hero  
.hero {  
position: relative;  
width: 100%;  
height: 100vh;  
overflow: hidden;
```

}

Este bloque define la cabecera principal de la página.

- **position: relative** permite colocar dentro de ella otros elementos con **position: absolute**.
 - **height: 100vh** hace que la cabecera ocupe toda la altura visible de la pantalla. El uso de unidades **vh** aparece en los apuntes como unidad viewport útil para diseño adaptable.
 - **overflow: hidden** evita que se vean partes del vídeo o de otros elementos fuera de la cabecera.
-

.hero video

```
.hero video {  
  position: absolute;  
  top: 50%;  
  left: 50%;  
  width: 110%;  
  height: 110%;  
  object-fit: cover;  
  transform: translate(-50%, -50%);  
  z-index: 1;  
  transition: 0.6s ease;  
  filter: brightness(0.80);  
}
```

Este bloque coloca el vídeo de fondo.

- **position: absolute** permite situarlo de forma precisa dentro de la cabecera.
- **top y left al 50%** lo colocan en el centro del contenedor.
- **transform: translate(-50%, -50%)** corrige el desplazamiento para centrarlo exactamente. La propiedad **transform** aparece en los apuntes de maquetación avanzada.

- `width: 110%` y `height: 110%` hacen que el vídeo sea un poco más grande que el contenedor para cubrir bien toda la cabecera.
- `object-fit: cover` hace que el vídeo rellene el espacio sin deformarse.
- `z-index: 1` controla el orden de apilamiento.
- `transition` suaviza cambios visuales.
- `filter: brightness(0.80)` oscurece un poco el vídeo para que pase a segundo plano.

Las propiedades `position`, `z-index`, `transform` y `transition` sí están dentro de lo que habéis visto en los PDF.

```
.overlay
.overlay {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: rgba(0, 0, 0, 0.35);
  z-index: 2;
  transition: 0.5s ease;
}
```

Este bloque crea una capa oscura sobre el vídeo para mejorar la legibilidad del texto.

- `rgba(...)` permite usar color con transparencia.
 - `position: absolute` y `width/height: 100%` hacen que cubra toda la cabecera.
 - `z-index` la coloca por encima del vídeo.
 - `transition` suaviza el cambio cuando se abre el menú.
-

5. Control del menú

#menu-toggle

```
#menu-toggle {  
  display: none;  
}
```

Ocultar el **checkbox** que se usa para controlar el menú. La idea es que el usuario no vea la casilla, sino que interactúe con el logo.

.menu-area

```
.menu-area {  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 120px;  
  z-index: 4;  
}
```

Crea un área invisible en la parte superior de la cabecera. Su función es permitir que el menú se abra al pasar el ratón por esa zona.

.logo-btn

```
.logo-btn {  
  position: absolute;  
  top: 25px;  
  left: 25px;  
  z-index: 6;  
  cursor: pointer;  
  transition: 0.3s ease;
```

```
}
```

Este bloque coloca el logotipo.

- `position: absolute` lo sitúa en una esquina.
- `cursor: pointer` indica que es interactivo.
- `transition` suaviza su comportamiento.

Hover del logo

```
.logo-btn:hover {  
  transform: scale(1.05);  
}
```

Aquí se usa `transform: scale(...)` para ampliar un poco el logo al pasar el ratón.

Se trata de un pequeño efecto visual para dar sensación de interactividad. La propiedad `transform` está contemplada en los apuntes.

6. Menú de navegación

.menu

```
.menu {  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 100%;  
  min-height: 120px;  
  padding: 40px 40px 40px 140px;  
  display: flex;  
  justify-content: flex-end;  
  align-items: center;  
  background: rgba(0, 0, 0, 0.18);  
  backdrop-filter: blur(6px);  
}
```

```
-webkit-backdrop-filter: blur(6px);
transform: translateX(-60px);
opacity: 0;
z-index: 5;
transition: 0.5s ease;
}
```

Este bloque da forma al menú.

- `position: absolute` permite situarlo encima del vídeo.
- `padding` genera separación interna. Esto se relaciona con el box model.
- `display: flex, justify-content` y `align-items` organizan los elementos del menú con Flexbox, algo que sí está explicado en los apuntes de maquetación.
- `background` añade un fondo semitransparente.
- `transform: translateX(-60px)` hace que inicialmente esté desplazado.
- `opacity: 0` lo deja oculto al principio.
- `transition` suaviza su aparición.

Lo menos “de clase” aquí es `backdrop-filter`, porque en los PDF no aparece de forma específica. Se puede justificar diciendo que se usa para dar un efecto de cristal difuminado, pero si quieres ir más seguro para EVAGD, podrías decir que se añadió como mejora visual personal.

`.menu ul`

```
.menu ul {
list-style: none;
display: flex;
gap: 30px;
flex-wrap: wrap;
}
```

- `list-style: none` quita los puntos de la lista.
 - `display: flex` alinea los enlaces en horizontal.
 - `gap` separa los elementos entre sí.
 - `flex-wrap: wrap` permite que bajen a otra línea si no caben.
- Todo esto encaja con Flexbox trabajado en clase.
-

Enlaces del menú

```
.menu ul li a {  
  text-decoration: none;  
  color: #ffffff;  
  text-transform: uppercase;  
  letter-spacing: 1px;  
  font-size: 0.9rem;  
  transition: 0.3s ease;  
}
```

```
.menu ul li a:hover {  
  color: #d5b78a;  
}
```

Aquí se da estilo a los enlaces:

- `text-decoration: none` quita el subrayado
 - `text-transform: uppercase` pone el texto en mayúsculas
 - `letter-spacing` separa un poco los caracteres, propiedad que también aparece en los apuntes.
 - `:hover` cambia el color al pasar el ratón
-

7. Texto de la cabecera

.hero-content

```
.hero-content {  
  position: absolute;  
  top: 50%;  
  left: 50%;  
  width: 90%;  
  max-width: 700px;  
  text-align: center;  
  transform: translate(-50%, -50%);  
  z-index: 4;  
  transition: 0.5s ease;  
}
```

Este bloque centra el contenido principal sobre el vídeo.

- **position: absolute** permite colocarlo encima del fondo
- **top**, **left** y **transform** lo centran
- **text-align: center** centra el texto
- **max-width** limita el ancho para que no quede demasiado extenso

Título y párrafo

```
.hero-content h1 { ... }  
.hero-content p { ... }
```

Aquí se controlan tamaño, separación y legibilidad del texto principal, usando propiedades básicas de texto como **font-size**, **line-height**, **margin** y **letter-spacing**, todas dentro del contenido de clase.

8. Reglas para abrir el menú

```
#menu-toggle:checked ~ .menu { ... }  
#menu-toggle:checked ~ .overlay { ... }  
#menu-toggle:checked ~ video { ... }  
#menu-toggle:checked ~ .hero-content { ... }
```

Estas reglas usan el selector de hermano general ~ y el pseudoestado **:checked**.

Su función es:

- mostrar el menú
- oscurecer más la capa
- aumentar un poco el vídeo
- reducir ligeramente la opacidad del texto

Esto permite crear interacción sin JavaScript.

Aquí sí hay una parte algo más avanzada, porque aunque el uso de pseudoclasas sí entra dentro de selectores y CSS, el truco del **checkbox** como controlador de menú no aparece de forma literal en los PDFs. Aun así, se puede justificar como una solución de CSS para interacción básica.

Reglas con **:hover**

```
.menu-area:hover ~ .menu { ... }  
.menu-area:hover ~ .overlay { ... }  
.menu-area:hover ~ video { ... }  
.menu-area:hover ~ .hero-content { ... }  
.menu:hover { ... }
```

Estas reglas permiten que el menú también se abra al pasar el ratón por la zona superior o sobre el propio menú.

Aquí se usa la pseudoclase **:hover**, que sí forma parte de los contenidos de selectores y pseudoclasas.

9. Contenido principal

main

```
main {  
  background-color: #f3eee6;  
}
```

Se usa para diferenciar visualmente la zona del contenido respecto a la cabecera.

.contenedor

```
.contenedor {  
  width: 90%;  
  max-width: 1200px;  
  margin: 0 auto;  
}
```

Este bloque sirve para centrar contenido y limitar su anchura.

margin: 0 auto se menciona en los apuntes como forma de centrar un elemento de bloque.

.subtitulo

```
.subtitulo {  
  display: inline-block;  
  margin-bottom: 12px;  
  font-size: 0.85rem;  
  letter-spacing: 2px;  
  text-transform: uppercase;  
  color: #8a6a45;  
}
```

Se usa para estilizar los subtítulos pequeños de las secciones.

10. Secciones con Flexbox

`.intro`

```
.intro {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  gap: 60px;  
  padding: 100px 5%;  
  background-color: #f3eee6;  
  color: #2d241b;  
}
```

Este bloque organiza la primera sección en dos columnas: texto e imagen.

- `display: flex` convierte la sección en contenedor flexible
- `justify-content` y `align-items` alinean los elementos
- `gap` separa columnas
- `padding` añade espacio interior

Todo esto encaja directamente con Flexbox visto en UD3.

`.intro-texto, .intro-imagen`

Estos bloques controlan el ancho de cada columna y hacen que el texto y la imagen queden equilibrados.

.intro-imagen img, .intro-imagen video

```
.intro-imagen img,  
.intro-imagen video {  
  width: 100%;  
  object-fit: cover;  
  border-radius: 18px;  
}
```

Aquí se da estilo común tanto a imágenes como a vídeos:

- **width: 100%** hace que ocupen el ancho del contenedor
 - **object-fit: cover** evita deformaciones
 - **border-radius** redondea las esquinas, una propiedad también explicada en los materiales.
-

11. Bloques de contenido

**.bloque-info, .bloque-destacado,
.bloque-experiencia, .cierre-home**

Estas clases definen cada sección del contenido principal, modificando:

- espaciado
- color de fondo
- color del texto
- tamaño de títulos
- ancho del contenido

Sirven para separar visualmente cada parte de la página y mantener coherencia.

12. Botón final

```
.boton-home {  
  display: inline-block;  
  padding: 14px 28px;  
  background-color: #2f2418;  
  color: #ffffff;  
  text-decoration: none;  
  border-radius: 999px;  
  transition: 0.3s ease;  
}
```

Este bloque crea el botón final:

- `display: inline-block` permite aplicar tamaño y padding
- `padding` genera espacio interior
- `border-radius: 999px` le da forma redondeada
- `transition` suaviza el cambio de color al hacer hover

13. Media queries

```
@media (max-width: 900px) { ... }  
@media (max-width: 600px) { ... }
```

Estas reglas permiten adaptar la web a tablets y móviles.
Se basan en el ancho máximo de pantalla, algo que aparece claramente en los apuntes sobre media queries y responsive.

Qué hacen:

- cambian el menú
- reducen tamaño del logo
- convierten secciones de dos columnas en una sola columna

- reducen tamaños de títulos y textos
- adaptan el espaciado general

Esto permite que la página sea más usable en diferentes dispositivos.

14. Qué partes son más avanzadas

Si necesitas dejar claro en EVAGD qué cosas no habías dado tanto, puedes decirlo así:

Las partes más avanzadas del CSS son:

- el uso del `checkbox` oculto para controlar el menú
- el uso de `transform` junto a `translate` y `scale` para animaciones de entrada
- el uso de `backdrop-filter` para un efecto visual de desenfoque
- la combinación de `position`, `z-index` y capas superpuestas en la cabecera

Aun así, varias de estas propiedades sí aparecen en los apuntes, especialmente `position`, `z-index`, `transform`, `transition`, Flexbox y media queries.

15. Conclusión

En resumen, el CSS se ha utilizado para:

- maquetar la página
- organizar cabecera, menú y secciones
- mejorar el aspecto visual de textos, imágenes y vídeos
- adaptar la web a diferentes tamaños de pantalla
- y crear una presentación más atractiva del contenido del Cenobio de Valerón

La mayor parte del código se puede justificar con los contenidos vistos en clase sobre modelo de caja, tipografías, imágenes, Flexbox, media queries, selectores, posicionamiento y transformaciones.